Práctica N°1: Medicion de Corriente con ajuste de PWM visualizado en OLED

Índice

1. Objetivos **2**
- 1.1. Objetivo General . . . 2
- 1.2. Objetivos Específicos . . . 2

2. Análisis y explicación del módulo Maestro **2**
- 2.1. Configuración e imágenes de los pines . . . 3
- 2.2. Configuración de periféricos (USART3) . . . 7
- 2.3. Funciones para el uso del módulo LoRa como maestro . . . 8
  - 2.3.1. Pulso de reset al RN2483 . . . 8
  - 2.3.2. Conversión a HEX ASCII (para radio tx) . . . 9
  - 2.3.3. Enviar y leer líneas por UART (polling) . . . 10
  - 2.3.4. Arranque y verificación del módulo . . . 11
  - 2.3.5. Transmisión básica en modo “radio” (punto a punto) . . . 11
  - 2.3.6. Integración en `main()` (fragmento inicial) . . . 14
  - 2.3.7. Diagnóstico de respuestas típicas . . . 15
- 2.4. Codigo y explicacion del main . . . 15
- 2.5. Codigo y explicacion del while . . . 17

3. Analisis y explicacion del modulo Esclavo **20**
- 3.1. Configuracion e imagenes de los pines . . . 20
- 3.2. Configuracion de perifericos (USART3 y SPI1) . . . 25
- 3.3. Funciones para la OLED (dentro de `main.c`) . . . 27
- 3.4. Funciones para el uso del modulo LoRa como esclavo . . . 30
- 3.5. Codigo y explicacion del main (Esclavo) . . . 35
- 3.6. Codigo y explicacion del while (Esclavo) . . . 38

4. Resultados **40**
- 4.1. Imágenes del funcionamiento . . . 41
- 4.2. Diferencias entre frecuencias (433 MHz vs 868 MHz) . . . 41

1. Objetivos

1.1. Objetivo General

Implementar un sistema de comunicación inalámbrica entre dos módulos *STM32F407G-DISC1* utilizando el protocolo **LoRa RN2483** a través de módulos Click Board, de manera que el nodo maestro pueda transmitir los datos al nodo esclavo, el cual los visualizará en una pantalla OLED SSD1351.

1.2. Objetivos Específicos

- Configurar los periféricos del *STM32F407G-DISC1* necesarios para el uso del módulo LoRa, tanto en el maestro como en el esclavo.
- Desarrollar funciones en lenguaje C para el control del módulo LoRa en modo maestro y esclavo.
- Implementar la visualización de los datos recibidos en la pantalla OLED SSD1351.
- Analizar el funcionamiento de la comunicación inalámbrica a diferentes frecuencias (433 MHz y 868 MHz).
- Comparar las ventajas y limitaciones del uso de LoRa en distintas aplicaciones de control y monitoreo.

2. Análisis y explicación del módulo Maestro

Ruta de trabajo de la Sección

En esta sección construiremos el nodo Maestro de forma incremental:

1. Definir y cablear los **pines** del módulo LoRa RN2483 en el *Discovery Shield*.
2. Configurar el **periférico USART3** en CubeMX para enlazar el STM32 con el RN2483.
3. (Más adelante) Implementar funciones de inicialización y envío de comandos al RN2483.
4. (Más adelante) Integrar el bucle principal para transmitir un dato (p. ej., PWM) al Esclavo.

2.1. Configuración e imágenes de los pines

Para el nodo Maestro usaremos únicamente los pines del módulo **LoRa RN2483** (Click Board) conectados al *STM32F407G-DISC1* mediante el *Discovery Shield*.

- **RST** → `PE10`
- **RTS** → `PB12`
- **CTS** → `PE7`
- **TX** → `PB11` (USART3_TX)
- **RX** → `PB10` (USART3_RX)

Notas :

- En el STM32F407, `PB10` y `PB11` usan la función alternativa **AF7** para **USART3** (TX/RX).
- `RST` (PE10) es salida GPIO para reiniciar el RN2483 (pulso bajo).
- `RTS` (PB12) y `CTS` (PE7) se tratarán como **GPIO** (no activaremos *hardware flow control* en USART3). Esto evita conflictos de AF y nos da control explícito desde software.

Cuadro 1: Mapa de pines del Maestro para el módulo LoRa RN2483

Señal RN2483	**Pin STM32**	**Tipo / Uso**
RST	PE10	GPIO salida (reset RN)
RTS	PB12	GPIO salida (flow control por software)
CTS	PE7	GPIO entrada (flow control por software)
TX	PB11	USART3_TX (AF7)
RX	PB10	USART3_RX (AF7)

A continuación, coloca las capturas del *CubeMX IOC* que muestren el pinout:

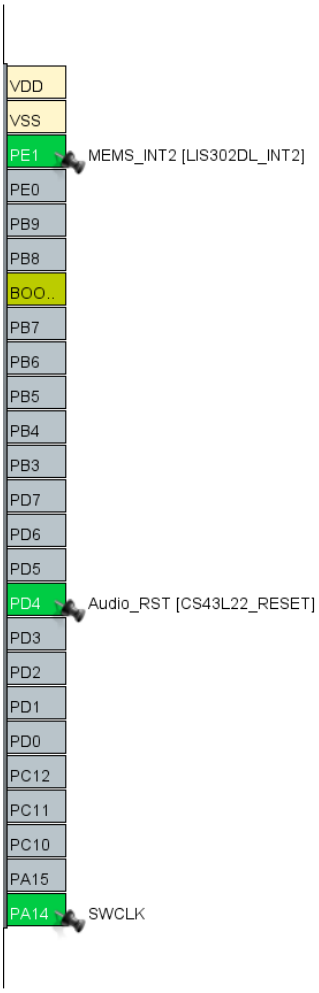


Figura 1: Vista Pinout & Configuration del Maestro: asignación de PB10/PB11 para USART3 y GPIO para RST/RTS/CTS.

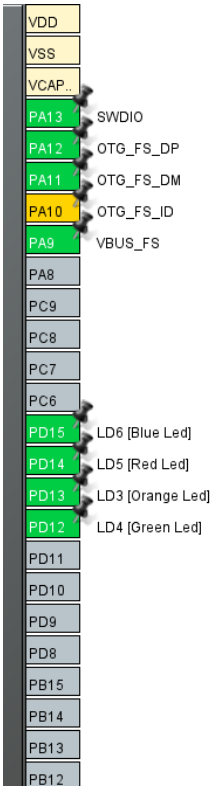


Figura 2: Vista Pinout & Configuration del Maestro: asignación de PB10/PB11 para USART3 y GPIO para RST/RTS/CTS.

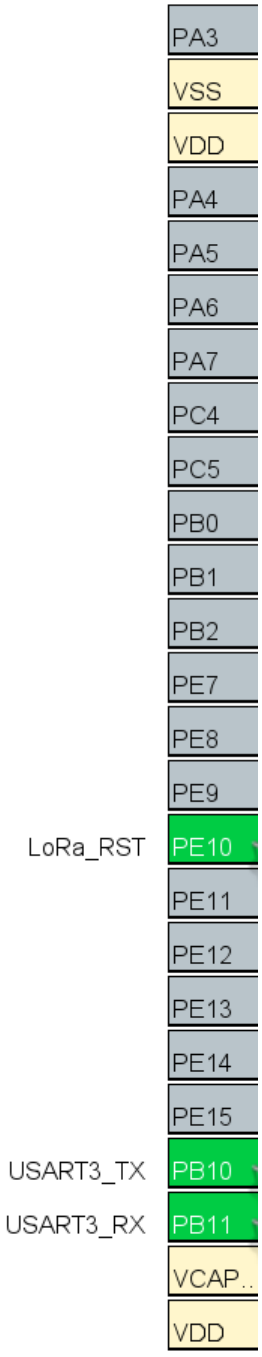


Figura 3: Vista Pinout & Configuration del Maestro: asignación de PB10/PB11 para USART3 y GPIO para RST/RTS/CTS.

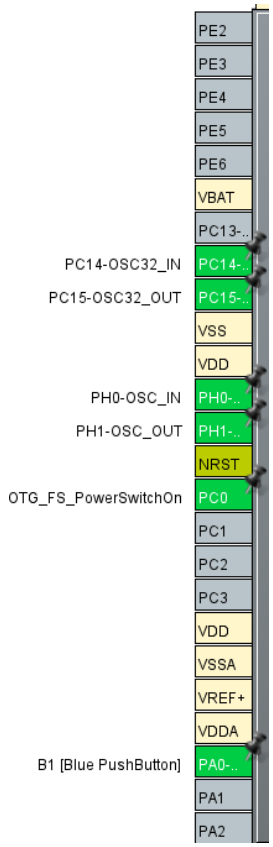


Figura 4: Vista Pinout & Configuration del Maestro: asignación de PB10/PB11 para USART3 y GPIO para RST/RTS/CTS.

2.2. Configuración de periféricos (USART3)

El RN2483 se comunica por UART a **57600 bps**, **8N1**, sin control de flujo por hardware. En CubeMX:

1. En **Pinout & Configuration** → **Connectivity** → **USART3**, marca **Asynchronous**.
2. En **Parameter Settings**:
   - **Baud Rate**: `57600`
   - **Word Length**: `8 Bits`
   - **Parity**: `None`
   - **Stop Bits**: `1`
   - **Mode**: `TX and RX`

- **Hardware Flow Control**: `None` (RTS/CTS por GPIO)
- **OverSampling**: `16`

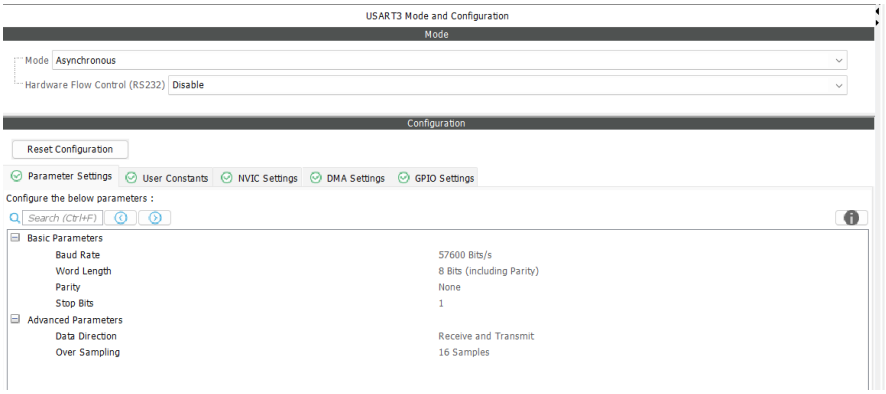


Figura 5: Parámetros de USART3 en CubeMX para el Maestro (57600 bps, 8N1, sin flujo por hardware).

2.3. Funciones para el uso del módulo LoRa como maestro

En esta subsección estudiamos las rutinas que permiten inicializar, configurar y transmitir con el módulo **RN2483** desde el Maestro. Todas las funciones están escritas para trabajar con **USART3** (PB10/PB11) y con **RST** en PE10. La comunicación se realiza en modo *radio* punto a punto (no LoRaWAN), enviando *payloads* en formato **HEX ASCII** (por ejemplo, el dígito `'1'` se transmite como `"31"`).

2.3.1. Pulso de reset al RN2483

El pulso de reset garantiza un arranque limpio y que el RN2483 emita su banner de inicio. Tras el pulso, es normal que el módulo tarde decenas de milisegundos en quedar listo.

```c
static inline void LORA_Reset_Pulse(void)
{
  // RST PE10: Low 10 ms, luego High
  HAL_GPIO_WritePin(GPIOE, GPIO_PIN_10, GPIO_PIN_RESET);
  HAL_Delay(10);
  HAL_GPIO_WritePin(GPIOE, GPIO_PIN_10, GPIO_PIN_SET);
  HAL_Delay(100); // esperar banner de inicio
}
```

Listing 1: Pulso de reset sobre RST (PE10)

- `GPIO_PIN_RESET` por 10 ms fuerza el reinicio del RN2483.
- La espera de `100 ms` da tiempo al módulo para inicializar su UART y banner de texto.

2.3.2. Conversión a HEX ASCII (para radio tx)

El comando `radio tx` del RN2483 exige que el *payload* vaya en **HEX ASCII**. Por eso convertimos cada byte a dos caracteres HEX.

```c
static void u8_to_hex2(uint8_t b, char out[3]) {
  const char *H = "0123456789ABCDEF";
  out[0] = H[(b >> 4) & 0x0F];
  out[1] = H[b & 0x0F];
  out[2] = '\0';
}
```

Listing 2: Conversión de un byte a dos dígitos HEX ASCII

Ejemplo: `0x31` → `"31"`; el carácter `'1'` (ASCII 0x31) se envía como `"31"`.

2.3.3. Enviar y leer líneas por UART (polling)

Trabajamos con funciones simples *línea-a-línea* para transmitir comandos y leer respuestas hasta \n, ignorando \r.

```
static int LORA_SendLine(const char *line, uint32_t
    timeout_ms)
{
  char tx[128];
  int n = snprintf(tx, sizeof(tx), "%s\r\n", line);
  if (n <= 0) return -1;
  if (HAL_UART_Transmit(&huart3, (uint8_t*)tx, (uint16_t
     )n, timeout_ms) != HAL_OK) return -2;
  return 0;
}
```

Listing 3: Enviar una línea con terminador CR+LF

```
static int LORA_ReadLine(char *out, size_t out_sz,
    uint32_t timeout_ms)
{
  size_t idx = 0;
  uint8_t ch;
  uint32_t start = HAL_GetTick();
  while ((HAL_GetTick() - start) < timeout_ms && idx < (
     out_sz-1))
  {
    if (HAL_UART_Receive(&huart3, &ch, 1, 10) == HAL_OK)
    {
      if (ch == '\n') break;
      if (ch != '\r') out[idx++] = ch;
    }
  }
  out[idx] = '\0';
  return (idx > 0) ? (int)idx : -1;
}
```

Listing 4: Leer una línea hasta \n con timeout

Claves didácticas:

- Siempre se envía `CR+LF` (\r\n) tras cada comando.
- `LORA_ReadLine` usa una ventana total `timeout_ms`, con lecturas de 10 ms para no bloquear.

- Si retorna `-1`, no llegó respuesta en el tiempo dado (revisar cableado/-baudios/reset/alimentación).

2.3.4. Arranque y verificación del módulo

Tras el *reset*, solicitamos la versión con `sys get ver`. Si esta etapa falla, no seguimos con TX.

```
static int LORA_BootAndCheck(void)
{
  char line[128];

  LORA_Reset_Pulse();

  // Algunos RN2483 envian un banner tras reset; intenta leerlo sin bloquearte mucho
  LORA_ReadLine(line, sizeof(line), 300);

  if (LORA_SendLine("sys get ver", 200) != 0) return -1;
  if (LORA_ReadLine(line, sizeof(line), 500) < 0) return -2;

  // Aqui podrias validar prefijo "RN"
  return 0;
}
```

Listing 5: Boot y chequeo de versión del RN2483

Sugerencia práctica: imprimir `line` por una UART de depuración o verlo en *Live Expressions* para confirmar que contiene algo como `RN2483 1.0.X`.

2.3.5. Transmisión básica en modo “radio” (punto a punto)

El modo *radio* funciona sin LoRaWAN y es ideal para pruebas P2P. Primero pausamos el stack MAC (`mac pause`), configuramos los parámetros físicos, y finalmente transmitimos.

```c
static int LORA_RadioBasicTx(const char *hex_payload)
{
  char cmd[64], resp[64];

  LORA_SendLine("mac pause", 200);
  LORA_ReadLine(resp, sizeof(resp), 500);   // suele devolver "4294967295"

  // Config LoRa basica @433 MHz
  LORA_SendLine("radio set mod lora", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  LORA_SendLine("radio set freq 433000000", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  LORA_SendLine("radio set pwr 14", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  LORA_SendLine("radio set sf sf7", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  // BW correcto para LoRa (125/250/500)
  LORA_SendLine("radio set bw 125", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  // Coding rate (4/5..4/8), CRC on, IQ invert off, preamble 8 (ajustables)
  LORA_SendLine("radio set cr 4/5", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  LORA_SendLine("radio set crc on", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  LORA_SendLine("radio set iqi off", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  LORA_SendLine("radio set prlen 8", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);

  // WDT 0 para evitar timeouts en radio
  LORA_SendLine("radio set wdt 0", 200);
  LORA_ReadLine(resp, sizeof(resp), 300);
```

```
41
42    // TX
43    snprintf(cmd, sizeof(cmd), "radio tx %s", hex_payload);
44    if (LORA_SendLine(cmd, 500) != 0) return -1;
45
46    if (LORA_ReadLine(resp, sizeof(resp), 500) < 0) return -2;    // esperado: "ok"
47    if (LORA_ReadLine(resp, sizeof(resp), 2000) < 0) return -3;  // esperado: "radio_tx_ok"
48    return 0;
49 }
```

Listing 6: TX básico punto a punto con configuración LoRa

Explicación parámetro por parámetro

- `mac pause`: deshabilita LoRaWAN y cede el control al modo `radio`. Devuelve un número (típicamente `4294967295`).
- `radio set mod lora`: define modulación LoRa (no FSK).
- `radio set freq ...`: **frecuencia en Hz**. Para 433 MHz: `433000000`. Para 868 MHz: `868000000`.
- `radio set pwr 14`: potencia de TX en dBm (típico `2..14` según región/módulo).
- `radio set sf sf7`: **Spreading Factor** (`sf7` a `sf12`). Mayor SF ⇒ mayor alcance y tiempo de aire.
- `radio set bw 125`: **Bandwidth** en kHz (`125`, `250`, `500`). Menor BW ⇒ mayor sensibilidad.
- `radio set cr 4/5`: **Coding Rate** (`4/5` .. `4/8`). Mayor redundancia ⇒ más robusto.
- `radio set crc on`: habilita CRC para detectar errores en payload.
- `radio set iqi off`: inversión de IQ; normalmente `off` para enlaces P2P estándar.
- `radio set prlen 8`: preámbulo; valores típicos `8..12`.

- `radio set wdt 0`: desactiva *watchdog* interno de radio (evita abortos por tiempo).

Cómo cambiar de 433 MHz a 868 MHz

```
/* 433 MHz */
LORA_SendLine("radio set freq 433000000", 200);

/* 868 MHz */
LORA_SendLine("radio set freq 868000000", 200);
```

Listing 7: Frecuencia de operación

Importante: Maestro y Esclavo deben usar **exactamente los mismos** parámetros de `freq`, `sf`, `bw`, `cr`, `crc`, `iqi` y `prlen` para recibir correctamente.

Ajustes recomendados por escenario

- **Prueba de laboratorio**: `sf7`, `bw 125`, `pwr 10..14`, `prlen 8`.
- **Alcance medio** (paredes/obstáculos): subir a `sf9..sf10` y/o `pwr 14`.
- **Máximo alcance**: `sf12`, `bw 125`, `pwr 14` (mayor tiempo de aire).

2.3.6. Integración en `main()` (fragmento inicial)

En el `main` inicializamos periféricos y comprobamos el RN2483. Si `LORA_BootAndCheck()` falla, encendemos un LED de error (o informamos por la consola).

```
/* USER CODE BEGIN 2 */
// Boot + version del LoRa
if (LORA_BootAndCheck() != 0) {
  // aqui puedes encender un LED o imprimir por otra
     UART para saber que fallo
}
/* USER CODE END 2 */
```

Listing 8: Fragmento didáctico del main: boot + verificación

Checklist rápido antes de probar TX

USART3 a 57600 bps (8N1) y cableado cruzado: `TX` → `RX`, `RX` → `TX`.

RST (PE10) configurado como salida; el pulso de reset funciona.

Alimentación y antena del RN2483 conectadas.

Mismo set de parámetros `radio set ...` en Maestro y Esclavo.

2.3.7. Diagnóstico de respuestas típicas

- **Tras** `radio tx`: primero `ok`, luego `radio_tx_ok`. Si aparece `radio_err`, revisar frecuencia/BW/SF o antena.
- **Sin respuesta**: revisar baudios (57600), pinout PB10/PB11 en AF7, masa común entre placa y módulo, y el pulso de `RST`.
- **Banner vacío**: ampliar el `HAL_Delay(100)` tras `RST` a `200..400 ms` en entornos ruidosos.

2.4. Codigo y explicacion del main

En esta parte dejamos listo el armazon del programa: inicializacion del HAL, reloj, GPIO, USB Host (opcional) y USART3. Tambien verificamos que el RN2483 responde antes de pasar al bucle principal (la logica del `while` se aborda en la siguiente subseccion).

Flujo general del `main`

1. **HAL_Init()** y **SystemClock_Config()** para preparar la MCU.
2. **MX_GPIO_Init()** para RST/RTS/CTS y LEDs de la placa.
3. **MX_USB_HOST_Init()** (*opcional* segun tu plantilla de proyecto).
4. **MX_USART3_UART_Init()** a 57600 bps (8N1).
5. **LORA_BootAndCheck()** para resetear y consultar `sys get ver`.
6. Si OK, encendemos LED verde; si falla, LED rojo.

```
1
2
3  int main(void)
4  {
5    /* USER CODE BEGIN 1 */
6    /* optional user init here */
7    /* USER CODE END 1 */
8
9    /* Reset of all peripherals, Initializes the Flash
       interface and the Systick. */
10   HAL_Init();
11
12   /* Configure the system clock */
13   SystemClock_Config();
14
15   /* Initialize all configured peripherals */
16   MX_GPIO_Init();
17   MX_USB_HOST_Init();       /* optional if your project
       uses USB Host */
18   MX_USART3_UART_Init();
19
20   /* USER CODE BEGIN 2 */
21   /* turn off LEDs initially (PD12..PD15 on STM32F407G-
       DISC1) */
22   HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12|GPIO_PIN_13|
       GPIO_PIN_14|GPIO_PIN_15, GPIO_PIN_RESET);
23
24   /* Boot + version check of RN2483 */
25   if (LORA_BootAndCheck() == 0) {
26     /* green LED = OK */
27     HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_SET);
28   } else {
29     /* red LED = FAIL */
30     HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
31   }
32   /* USER CODE END 2 */
33
34
35 }
```

Listing 9: Maestro: main antes del while (sin SPI ni TIM)

Explicacion didactica, linea por linea (resumen)

- `HAL_Init();` inicializa la capa HAL y el SysTick. Sin esto, los `HAL_Delay()` no funcionan.
- `SystemClock_Config();` define la fuente y multiplicadores del reloj. Mantener estable para evitar drift en UART.
- `MX_GPIO_Init();` configura:
  - **PE10** como salida para `RST` del RN2483.
  - **PB12** como salida para `RTS` (software).
  - **PE7** como entrada para `CTS` (software).
  - LEDs **PD12..PD15** para diagnostico visual.
- `MX_USB_HOST_Init();` es opcional; si tu proyecto plantilla ya lo incluye, lo mantenemos.
- `MX_USART3_UART_Init();` coloca la UART a 57600 bps, 8N1, sin hardware flow control (coincide con RN2483).
- `LORA_BootAndCheck();` hace *reset* del modulo y envia `"sys get ver"`. Si responde, LED verde; si no, LED rojo.

Checklist rapido de esta etapa

☐ USART3 activo a 57600 bps (AF7 en PB10/PB11).

☐ RST en PE10 funcionando (pulso bajo + retardo).

☐ LED verde enciende si hay respuesta del RN2483; rojo si falla.

2.5. Codigo y explicacion del while

El bucle principal realiza un envio periodico de un dato simple (un digito ASCII 0..9) al Esclavo. El payload se manda en **HEX ASCII** porque el comando `radio tx` del RN2483 lo exige. Por ejemplo, el caracter `'1'` (ASCII 0x31) se transmite como `"31"`.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* Mantener tareas de USB Host (si aplica en tu
     plantilla) */
  MX_USB_HOST_Process();

  /* Envio periodico cada 1000 ms */
  static uint8_t dig = 0;          // digito actual
     (0..9)
  static uint32_t t_tx = 0;
  const uint32_t TX_PERIOD_MS = 1000;

  if (HAL_GetTick() - t_tx >= TX_PERIOD_MS)
  {
    t_tx = HAL_GetTick();

    /* Convertir el digito '0'..'9' a su codigo ASCII y
       luego a HEX ASCII */
    char payload_hex[3];                              //
        2 hex + terminador
    uint8_t ascii_digit = (uint8_t)('0' + dig);       //
        '0'..'9'
    u8_to_hex2(ascii_digit, payload_hex);             //
        ej. '1' -> "31"

    /* Transmitir por LoRa en modo radio (punto a punto)
        */
    int r = LORA_RadioBasicTx(payload_hex);           //
        envia "radio tx 30..39"
    if (r == 0) {
      /* LED verde = TX OK */
      HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12);
      /* apagar LED rojo por si estaba encendido */
      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14,
         GPIO_PIN_RESET);
    } else {
      /* LED rojo = fallo de TX */
      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET
         );
    }
```

```
33
34      /* siguiente digito 0..9 */
35      dig++;
36      if (dig > 9) dig = 0;
37    }
38 }
39 /* USER CODE END WHILE */
```

Listing 10: While del Maestro: envio periodico 0..9 en HEX ASCII

Que esta pasando paso a paso

- **Scheduler por millis**: se usa `HAL_GetTick()` para disparar el envio cada `TX_PERIOD_MS`. Sin `HAL_Delay` bloqueante, el bucle queda fluido.
- **Dato enviado**: el Maestro no manda el numero binario `0..9`, sino el **caracter ASCII** `'0'..'9'`. Ese byte ASCII se convierte a **HEX ASCII** con `u8_to_hex2` para ajustarse a `radio tx`.
- **Diagnostico visual**: si el envio termina en `radio_tx_ok`, se conmuta el LED verde (PD12). En error, se enciende el LED rojo (PD14).
- **Periodo de envio**: ajusta `TX_PERIOD_MS` (p.ej., 200 para 5 Hz o 5000 para 0.2 Hz).

Como enviar un byte arbitrario (p.ej., PWM 0..255) Si en lugar de un caracter `'0'..'9'` quieres mandar un valor de 0..255:

```
1 uint8_t pwm_byte = 127;      /* ejemplo */
2 char payload_hex[3];          /* "00".."FF" + '\0' */
3 u8_to_hex2(pwm_byte, payload_hex);
4 (int) LORA_RadioBasicTx(payload_hex);
```

Como enviar dos bytes (p.ej., un valor de 10..bit) Concatena dos pares HEX:

```
uint16_t value = 1023;       /* ejemplo 10-bit */
char hex_hi[3], hex_lo[3], payload_hex[5]; /* "HHLL" + '\0' */
u8_to_hex2((uint8_t)(value >> 8), hex_hi);
u8_to_hex2((uint8_t)(value & 0xFF), hex_lo);
snprintf(payload_hex, sizeof(payload_hex), "%s%s", hex_hi, hex_lo);
/* Ej: 0x03FF -> "03FF" */
(int) LORA_RadioBasicTx(payload_hex);
```

3. Analisis y explicacion del modulo Esclavo

Que haremos en el Esclavo

El Esclavo recibira por **LoRa RN2483** los datos enviados por el Maestro en modo *radio* (punto a punto), decodificara el *payload* en **HEX ASCII** (p. ej., "30".."39representa '0'..'9') y los mostrara en la **OLED SSD1351**.

3.1. Configuracion e imagenes de los pines

Pines del RN2483 (LoRa) en el Esclavo Para homogeneidad con el Maestro, mantenemos el mismo mapeo:

- **RST** → `PE10`
- **RTS** → `PB12`
- **CTS** → `PE7`
- **TX** → `PB11` (USART3_TX)
- **RX** → `PB10` (USART3_RX)

Cuadro 2: Mapa de pines del Esclavo para LoRa RN2483

Senal RN2483	**Pin STM32**	**Tipo / Uso**
RST	PE10	GPIO salida (reset RN)
RTS	PB12	GPIO salida (flow control por software)
CTS	PE7	GPIO entrada (flow control por software)
TX	PB11	USART3_TX (AF7)
RX	PB10	USART3_RX (AF7)

Pines de la OLED SSD1351 en el Esclavo Usaremos **SPI1** y las siguientes lineas de control:

- **SPI1 SCK** → `PB3`
- **SPI1 MISO** → `PB4`
- **SPI1 MOSI** → `PB5`
- **EN** → `PD2` (habilitacion del modulo)
- **RES** → `PE4` (reset OLED)
- **CS** → `PE5` (chip select)
- **RW** → `PC5` (read/write; normalmente en escritura)
- **D/C** → `PB1` (data/command)

Cuadro 3: Mapa de pines del Esclavo para OLED SSD1351 (SPI1)

Senal OLED	**Pin STM32**	**Tipo / Uso**
SCK	PB3	SPI1_SCK
MISO	PB4	SPI1_MISO
MOSI	PB5	SPI1_MOSI
EN	PD2	GPIO salida (enable modulo)
RES	PE4	GPIO salida (reset OLED)
CS	PE5	GPIO salida (chip select)
RW	PC5	GPIO salida (modo escritura recomendado)
D/C	PB1	GPIO salida (seleccion dato/comando)

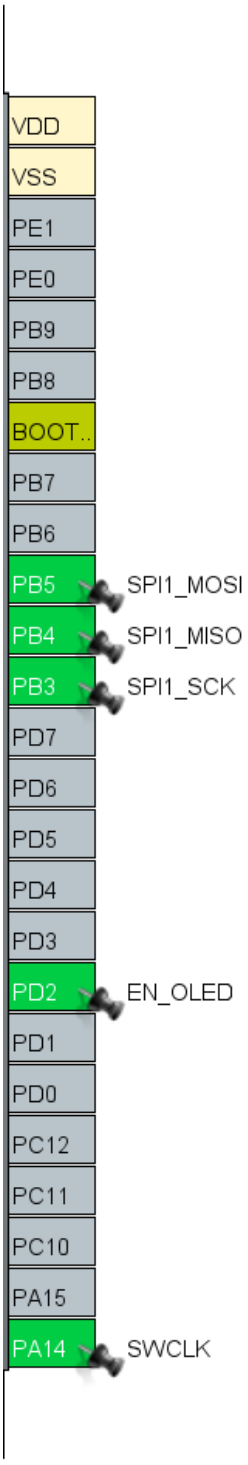


Figura 6: Esclavo: vista general de Pinout & Configuration.

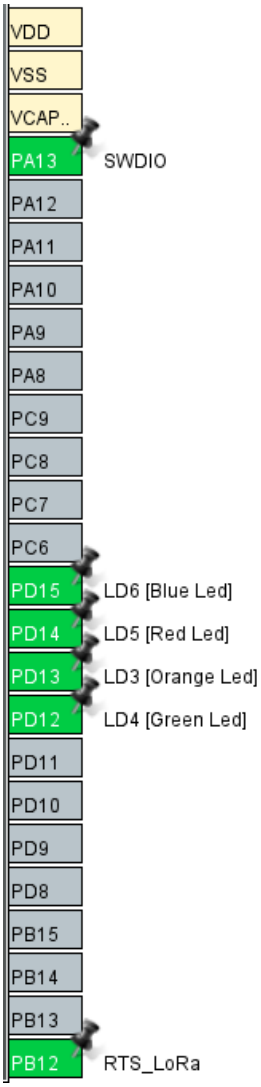


Figura 7: Esclavo: vista general de Pinout & Configuration.

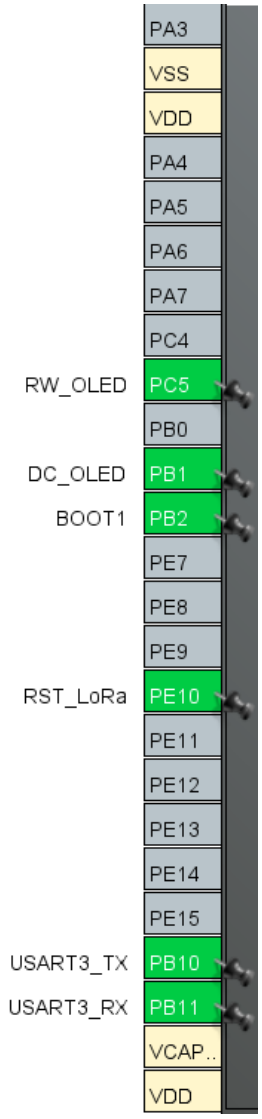


Figura 8: Esclavo: vista general de Pinout & Configuration.

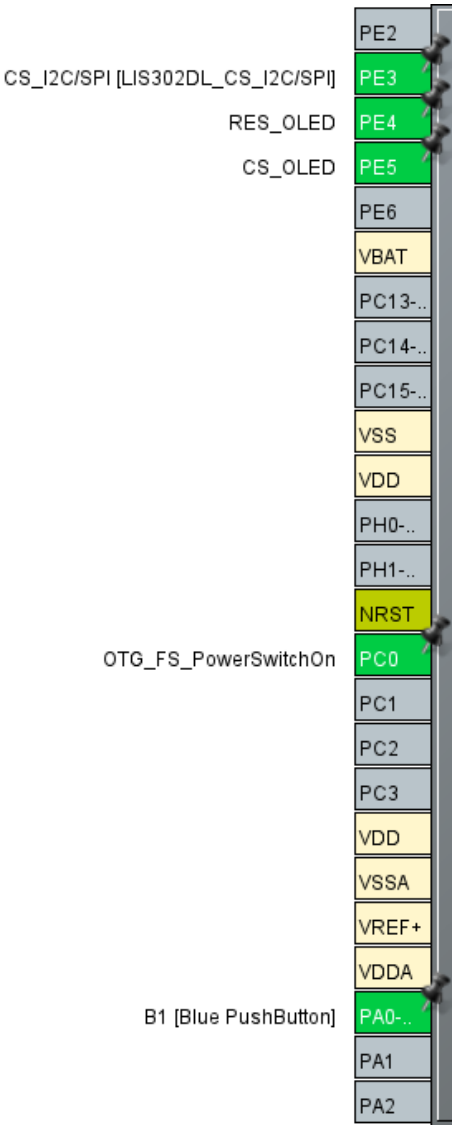


Figura 9: Esclavo: vista general de Pinout & Configuration.

3.2. Configuracion de perifericos (USART3 y SPI1)

USART3 (LoRa) El Esclavo usara **USART3** a **57600 bps**, **8N1**, **sin** control de flujo por hardware:

1. **Pinout & Configuration → Connectivity → USART3**: **Asynchronous**.
2. **Parameter Settings**: `BaudRate=57600`, `WordLength=8B`, `Parity=None`, `StopBits=1`, `Mode=TX&RX`, `HwFlowCtl=None`, `OverSampling=16`.

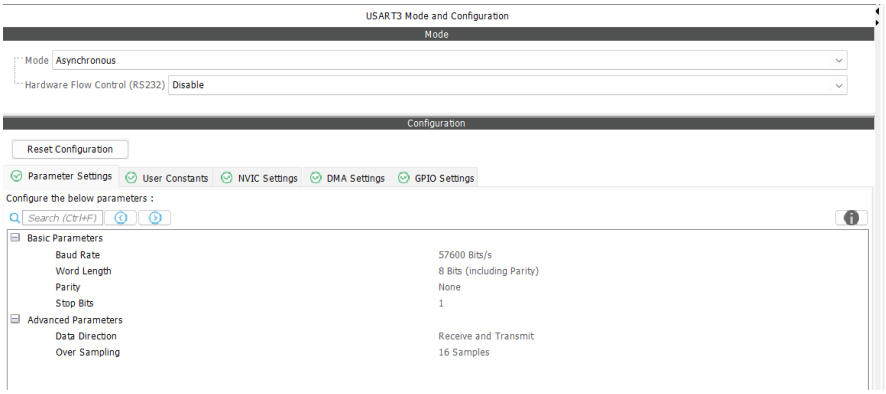


Figura 10: USART3 en CubeMX para Esclavo (57600 bps, 8N1, sin flujo por hardware).

SPI1 (OLED) La OLED SSD1351 se comunica por **SPI1** como **Master, 8 bits**. Usa la configuracion que espera `ssd1351.c` (tu libreria). Un punto de partida comun es **SPI modo 0** (`CPOL=Low`, `CPHA=1Edge`). Si la imagen no aparece o muestra ruido, prueba **modo 3** (`CPOL=High`, `CPHA=2Edge`).

1. **Pinout & Configuration** → **Connectivity** → **SPI1**: **Full Duplex Master**.
2. **Parameter Settings** (sugeridos):
   - **Mode**: Master, **Direction**: 2Lines
   - **Data Size**: 8 Bits, **First Bit**: MSB First
   - **Clock Polarity**: Low, **Clock Phase**: 1 Edge (*modo 0*)
   - **NSS**: Soft
   - **BaudRate Prescaler**: 16 ($\approx$ 5.25 MHz con APB2 a 84 MHz)
   - **TI Mode**: Disabled, **CRC**: Disabled

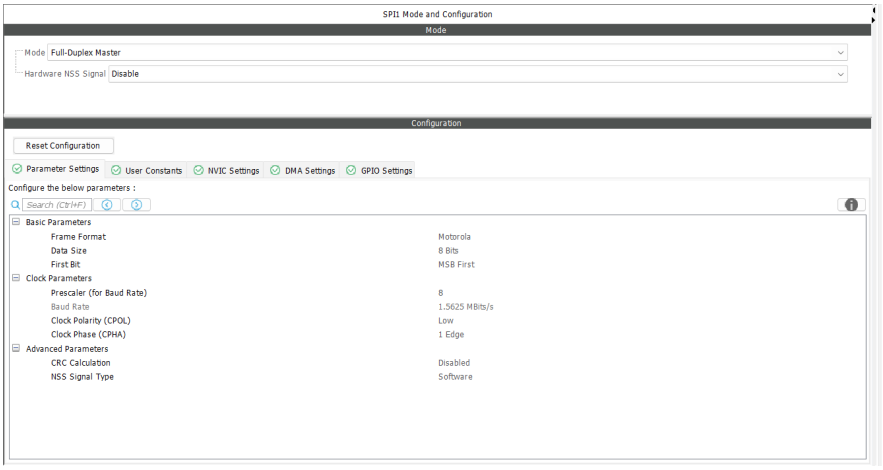


Figura 11: SPI1 en CubeMX para Esclavo (SSD1351). Ajustar CPOL/CPHA segun libreria.

3.3. Funciones para la OLED (dentro de `main.c`)

En esta practica **reutilizamos la misma libreria** `ssd1351.c/.h` implementada en la practica anterior. Por lo tanto, **no reescribimos** la logica de bajo nivel (comandos, set column/row, protocolos, fuentes, etc.). Aqui solo dejamos claras las **exportaciones** necesarias y las **funciones minimas** en `main.c` para encender la pantalla, inicializarla y mostrar datos.

Exportaciones necesarias en `main.c`

- `#include "ssd1351.h"` para acceder a `SSD1351_Init()`, `SSD1351_Clear()`, `SSD1351_DrawText()`, etc.
- `extern SPI_HandleTypeDef hspi1;` la libreria usa `hspi1` (SPI1) para transferencias.
- Asegurar que las **macros de pines** de la OLED usadas por la libreria (`OLED_CS_Pin`, `OLED_DC_Pin`, `OLED_RES_Pin`, `OLED_EN_Pin`, `OLED_RW_Pin`) **coinciden** con el mapeo de esta practica: PB3=SCK, PB4=MISO, PB5=MOSI, PD2=EN, PE4=RES, PE5=CS, PC5=RW, PB1=DC.

```c
/* USER CODE BEGIN Includes */
#include "ssd1351.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
extern SPI_HandleTypeDef hspi1;  /* usado por la
   libreria ssd1351 */
/* USER CODE END PV */
```

Listing 11: Includes y exports basicos para OLED en main.c

Secuencia de encendido + init (usar libreria anterior) La libreria ya realiza la inicializacion del controlador SSD1351. Antes de llamarla, recomendamos una secuencia simple de *enable* y *reset* desde GPIO para garantizar un arranque limpio.

```c
/* USER CODE BEGIN 2 */
/* habilitar modulo (si aplica en tu shield) */
HAL_GPIO_WritePin(OLED_EN_GPIO_Port, OLED_EN_Pin,
   GPIO_PIN_SET);

/* asegurarse de CS alto antes de configurar SPI */
HAL_GPIO_WritePin(OLED_CS_GPIO_Port, OLED_CS_Pin,
   GPIO_PIN_SET);

/* seleccionar escritura (si RW se usa) */
HAL_GPIO_WritePin(OLED_RW_GPIO_Port, OLED_RW_Pin,
   GPIO_PIN_RESET); /* write */

/* reset OLED */
HAL_GPIO_WritePin(OLED_RES_GPIO_Port, OLED_RES_Pin,
   GPIO_PIN_RESET);
HAL_Delay(10);
HAL_GPIO_WritePin(OLED_RES_GPIO_Port, OLED_RES_Pin,
   GPIO_PIN_SET);
HAL_Delay(100);

/* inicializacion via libreria de la practica anterior
   */
```

```
SSD1351_Init();
SSD1351_Clear();

/* mensaje de prueba */
SSD1351_DrawText(20, 0, "OLED␣OK", 0xFFFF, 0x0000, 2);
/* USER CODE END 2 */
```

Listing 12: Power-up basico y llamada a la libreria

Helpers minimos en `main.c` para esta practica Como ejemplo didactico, dejamos una funcion para **mostrar el ultimo digito recibido** (el Esclavo decodifica el payload y pasa el caracter que desea pintar). Todo el dibujo real lo hace la libreria (`DrawText`, colores RGB565, etc.).

```
/* USER CODE BEGIN 0 */
static void OLED_ShowLastDigit(char c)
{
  /* etiqueta y valor en zona libre */
  SSD1351_DrawText(20, 66, "D:", 0xFFFF, 0x0000, 2);
  char s[2] = { c, '\0' };
  SSD1351_DrawText(50, 66, s, 0xFFE0, 0x0000, 2);
}
/* USER CODE END 0 */
```

Listing 13: Helper: mostrar un digito en la SSD1351

Puntos a verificar para alinear libreria y hardware

- **SPI1**: PB3=SCK, PB4=MISO, PB5=MOSI en AF5. Parametros tipicos: 8-bit, Master, NSS=Soft, modo 0 (CPOL Low, CPHA 1Edge). Si la imagen no aparece o sale ruido, probar modo 3 (CPOL High, CPHA 2Edge).
- **GPIO de control**: EN=PD2 (alto), RES=PE4 (pulso bajo), CS=PE5 (alto inactivo), RW=PC5 (low para escribir), DC=PB1 (0=cmd, 1=data).
- **Consistencia de macros**: que `ssd1351.c` use las mismas macros que definimos en esta guia.
- **Orden de llamadas**: primero GPIO (EN/RES/CS/RW), luego `MX_SPI1_Init()`, y finalmente `SSD1351_Init()`.

Uso tipico desde `main()`

```
/* USER CODE BEGIN 2 */
SSD1351_Init();
SSD1351_Clear();
SSD1351_DrawText(10, 10, "RX READY", 0x07E0, 0x0000, 1);
/* USER CODE END 2 */

/* ... en el while, cuando decodificas el payload ... */
OLED_ShowLastDigit('3');  /* ejemplo */
/* Tambien puedes imprimir diagnostico */
SSD1351_DrawText(0, 80, "RN: radio_rx 30", 0xFFFF, 0
   x0000, 1);
```

Listing 14: Uso sintetico en main(): init y mensaje

3.4. Funciones para el uso del modulo LoRa como esclavo

El Esclavo debe: (1) resetear el RN2483, (2) pausar el stack MAC, (3) aplicar la misma configuracion de radio que el Maestro (freq/SF/BW/CR/CRC/IQI/PRLEN), (4) entrar en modo escucha con `radio rx 0`, y (5) procesar eventos: `radio_rx <hex>`, `radio_err` o `busy`. Los helpers de UART LORA_SendLine/LORA_ReadLine ya vistos se reutilizan tal cual.

Reset del RN2483 (GPIO PE10)

```
static inline void LORA_Reset_Pulse(void)
{
  HAL_GPIO_WritePin(LORA_RST_GPIO_Port, LORA_RST_Pin,
     GPIO_PIN_RESET);
  HAL_Delay(10);
  HAL_GPIO_WritePin(LORA_RST_GPIO_Port, LORA_RST_Pin,
     GPIO_PIN_SET);
  HAL_Delay(300); /* tiempo para banner y UART lista */
}
```

Listing 15: Pulso de reset al RN2483

Aplicar configuracion PHY por defecto (parametros de radio) La siguiente rutina aplica un set solido (igual al Maestro), dejando la **frecuencia** como argumento para que puedas conmutar entre 433 MHz y 868 MHz con una sola linea.

```
static int LORA_ApplyDefaultPhy(uint32_t freq_hz)
{
  char resp[96], cmd[48];

  /* ceder control al modo radio (no LoRaWAN) */
  LORA_SendLine("mac␣pause", 200);
  LORA_ReadLine(resp, sizeof(resp), 500); /* tipico:
     4294967295 */

  /* mod LoRa y parametros base */
  LORA_SendLine("radio␣set␣mod␣lora", 200);
     LORA_ReadLine(resp, sizeof(resp), 300);

  snprintf(cmd, sizeof(cmd), "radio␣set␣freq␣%lu", (
     unsigned long)freq_hz);
  LORA_SendLine(cmd, 200);
     LORA_ReadLine(resp, sizeof(resp), 300);

  LORA_SendLine("radio␣set␣pwr␣14", 200);
     LORA_ReadLine(resp, sizeof(resp), 300);
  LORA_SendLine("radio␣set␣sf␣sf7", 200);
     LORA_ReadLine(resp, sizeof(resp), 300);
  LORA_SendLine("radio␣set␣bw␣125", 200);
     LORA_ReadLine(resp, sizeof(resp), 300);
```

```
18   LORA_SendLine("radio set cr 4/5", 200);
        LORA_ReadLine(resp, sizeof(resp), 300);
19   LORA_SendLine("radio set crc on", 200);
        LORA_ReadLine(resp, sizeof(resp), 300);
20   LORA_SendLine("radio set iqi off", 200);
        LORA_ReadLine(resp, sizeof(resp), 300);
21   LORA_SendLine("radio set prlen 8", 200);
        LORA_ReadLine(resp, sizeof(resp), 300);
22   LORA_SendLine("radio set wdt 0", 200);
        LORA_ReadLine(resp, sizeof(resp), 300);
23
24   return 0;
25 }
```

Listing 16: Configurar PHY LoRa basica para Esclavo

Entrar en escucha continua (`radio rx 0`)

```
1  static int LORA_InitAndListen(uint32_t freq_hz)
2  {
3    char resp[96];
4
5    LORA_Reset_Pulse();                              /*
        reset fisico */
6    LORA_SendLine("sys get ver", 300);               /*
        opcional: banner/version */
7    LORA_ReadLine(resp, sizeof(resp), 1000);         /* no
        critico si no llega */
8
9    if (LORA_ApplyDefaultPhy(freq_hz) != 0) return -1;
10
11   if (LORA_SendLine("radio rx 0", 200) != 0) return -2;
12   if (LORA_ReadLine(resp, sizeof(resp), 500) < 0) return
        -3; /* "ok" */
13   HAL_Delay(20); /* evita "busy" inmediato */
14   return 0;
15 }
16
17 /* atajos comodos */
18 static inline int LORA_InitAndListen_433(void) { return
     LORA_InitAndListen(433000000UL); }
19 static inline int LORA_InitAndListen_868(void) { return
     LORA_InitAndListen(868000000UL); }
```

Listing 17: Inicializar y quedar escuchando

Rearmar la recepcion tras cada evento El RN2483 requiere rearmar `radio rx 0` para seguir escuchando despues de cada evento (rx, err, busy).

```
static void LORA_RearmRx(void)
{
  char resp[32];
  LORA_SendLine("radio␣rx␣0", 200);
  LORA_ReadLine(resp, sizeof(resp), 500); /* "ok" */
  HAL_Delay(20);
}
```

Listing 18: Rearmar el receptor

Decodificar el primer byte del payload (HEX ASCII) Los payloads del Maestro se envian en HEX ASCII (p. ej., '0' -¿"30"). Esta funcion intenta extraer el **primer byte** del mensaje `radio_rx <hex>`.

```
static int hex2_to_u8(const char *h, uint8_t *out)
{
  uint8_t v = 0;
  for (int i = 0; i < 2; i++) {
    char c = h[i];
    uint8_t d;
    if      (c >= '0' && c <= '9') d = (uint8_t)(c - '0'
       );
    else if (c >= 'A' && c <= 'F') d = (uint8_t)(10 + (c
        - 'A'));
    else if (c >= 'a' && c <= 'f') d = (uint8_t)(10 + (c
        - 'a'));
    else return -1;
    v = (uint8_t)((v << 4) | d);
  }
  *out = v;
  return 0;
}
```

Listing 19: HEX ASCII -¿byte (primer par)

```c
static int LORA_TryDecodeFirstByte(const char *line,
    uint8_t *out_byte)
{
  /* espera "radio_rx <hex...>" */
  if (strncmp(line, "radio_rx ", 9) != 0) return 0; /*
     no es un paquete */

  const char *p = line + 9;
  while (*p == ' ') p++;              /* saltar espacios
     */
  if (!isxdigit((unsigned char)p[0]) || !isxdigit((
     unsigned char)p[1])) return -1;

  return hex2_to_u8(p, out_byte);     /* devuelve 0 si OK
      */
}
```

Listing 20: Intentar decodificar el primer byte de radio$_r x$...”

Poll no bloqueante: leer una linea y manejar eventos Este helper lee una linea del RN (con timeout corto) y, si hay paquete, **devuelve 1 y el byte decodificado**. Si no hay nada, devuelve **0**. En errores, **-1**. La rearmada de RX se hace internamente.

```c
static int LORA_PollAndGetByte(uint8_t *out_byte,
    uint32_t read_timeout_ms)
{
  char line[96];
  int n = LORA_ReadLine(line, sizeof(line),
     read_timeout_ms);
  if (n <= 0) return 0;        /* no hubo linea en este
     ciclo */

  /* opcional: aqui puedes dibujar 'line' en OLED para
     diagnostico */

  if (!strcmp(line, "radio_err") || !strcmp(line, "busy"
     )) {
    LORA_RearmRx();
    return 0;
  }

  uint8_t b;
```

```
15    int r = LORA_TryDecodeFirstByte(line, &b);
16    if (r == 0) {
17      /* paquete valido: rearmar y devolver el byte */
18      LORA_RearmRx();
19      *out_byte = b;
20      return 1;
21    } else if (r < 0) {
22      /* linea "radio_rx ..." pero mal formada: rearmar */
23      LORA_RearmRx();
24      return -1;
25    }
26
27    /* otra linea informativa; no es paquete de datos */
28    return 0;
29  }
```

Listing 21: Poll y manejo de eventos LoRa en Esclavo

Como elegir la frecuencia (433 vs 868 MHz) El Esclavo debe **igualar** la PHY del Maestro:

```
1  /* 433 MHz */
2  if (LORA_InitAndListen_433() != 0) { /* manejo de error
       */ }
3
4  /* 868 MHz */
5  // if (LORA_InitAndListen_868() != 0) { /* manejo de
       error */ }
```

Listing 22: Elegir banda al iniciar

3.5. Codigo y explicacion del main (Esclavo)

En el `main()` inicializamos la MCU, GPIO, USART3 y SPI1; encendemos e inicializamos la **OLED SSD1351** con la libreria ya usada en la practica anterior; y dejamos el **RN2483** escuchando con `radio rx 0` (banda seleccionable 433/868 MHz). El bucle `while` se desarrolla en la siguiente subseccion.

Flujo general

1. **HAL_Init()** y **SystemClock_Config()**.
2. **MX_GPIO_Init()** para LoRa (RST/RTS/CTS), OLED (EN/RES/CS/RW/DC) y LEDs.

3. **MX_USART3_UART_Init()** a 57600 bps (8N1, sin HW flow).
4. **MX_SPI1_Init()** (modo master, 8 bits) para la OLED.
5. **OLED power-up** (EN/RES/CS/RW) y **SSD1351_Init()** de la libreria.
6. **LoRa Init & Listen** con misma PHY que el Maestro.

```
1
2  int main(void)
3  {
4    /* USER CODE BEGIN 1 */
5    /* user pre-init (opcional) */
6    /* USER CODE END 1 */
7
8    HAL_Init();
9    SystemClock_Config();
10
11   MX_GPIO_Init();
12   MX_USART3_UART_Init();
13   MX_SPI1_Init();
14
15   /* USER CODE BEGIN 2 */
16   /* apagar LEDs inicialmente */
17   HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12|GPIO_PIN_13|
       GPIO_PIN_14|GPIO_PIN_15, GPIO_PIN_RESET);
18
19   /* --- OLED power-up + init (reutiliza tu libreria
       ssd1351) --- */
20   HAL_GPIO_WritePin(OLED_EN_GPIO_Port,  OLED_EN_Pin,
       GPIO_PIN_SET);    /* enable */
21   HAL_GPIO_WritePin(OLED_CS_GPIO_Port,  OLED_CS_Pin,
       GPIO_PIN_SET);    /* CS high idle */
22   HAL_GPIO_WritePin(OLED_RW_GPIO_Port,  OLED_RW_Pin,
       GPIO_PIN_RESET);  /* write */
23   HAL_GPIO_WritePin(OLED_RES_GPIO_Port, OLED_RES_Pin,
       GPIO_PIN_RESET);  /* reset */
24   HAL_Delay(10);
25   HAL_GPIO_WritePin(OLED_RES_GPIO_Port, OLED_RES_Pin,
       GPIO_PIN_SET);
26   HAL_Delay(100);
27
```

```
28    SSD1351_Init();                /* libreria de la
          practica anterior */
29    SSD1351_Clear();
30    SSD1351_DrawText(10, 0,  "OLED␣OK",  0xFFFF, 0x0000,
          1);
31    SSD1351_DrawText(10, 16, "LoRa␣RX...",0x07E0, 0x0000,
          1);
32
33    /* --- LoRa: inicializar y quedar escuchando --- */
34    /* Elige banda segun tu prueba: 433 MHz o 868 MHz.
35       Asegurate de que el Maestro usa la misma
             configuracion. */
36
37    if (LORA_InitAndListen_433() == 0) {
38      SSD1351_DrawText(10, 32, "LoRa␣RX␣OK", 0x07E0, 0
            x0000, 1);
39      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_SET);
              /* LED verde */
40    } else {
41      SSD1351_DrawText(10, 32, "LoRa␣FAIL",  0xF800, 0
            x0000, 1);
42      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
              /* LED rojo */
43      /* puedes quedarte aqui o reintentar mas adelante */
44    }
45    /* USER CODE END 2 */
46
47    /* Infinite loop */
48    /* USER CODE BEGIN WHILE */
49
50  }
```

Listing 23: Esclavo: main antes del while (OLED + LoRa)

Explicacion didactica (resumen)

- **SPI1 (OLED)**: PB3=SCK, PB4=MISO, PB5=MOSI (AF5). NSS=Soft. Modo sugerido: 0 (CPOL Low, CPHA 1Edge); si ves ruido, probar modo 3.
- **GPIO OLED**: EN=PD2 (alto), RES=PE4 (pulso bajo), CS=PE5 (alto inactivo), RW=PC5 (low para escribir), DC=PB1 (0=cmd, 1=data).

- **USART3 (LoRa)**: PB10=RX, PB11=TX (AF7), 57600 bps, 8N1, sin HW flow. RST=PE10 con pulso de 10 ms + 300 ms de espera.
- **Init LoRa**: se pausa MAC (`mac pause`), se aplica PHY igual al Maestro (`freq/sf/bw/cr/crc/iqi/prlen`) y se arma `radio rx 0`.

Cambiar rapidamente de 433 a 868 MHz

```
/* 433 MHz por defecto */
if (LORA_InitAndListen_433() != 0) { /* manejo de error */ }

/* Para 868 MHz, usa esto en su lugar */
// if (LORA_InitAndListen_868() != 0) { /* manejo de error */ }
```

3.6. Codigo y explicacion del while (Esclavo)

El bucle principal consulta periodicamente al RN2483 sin bloquear, decodifica el *payload* (primer byte en HEX ASCII) y actualiza la OLED. Si llega un paquete valido, parpadea el LED verde; si hay una linea mal formada, dejamos encendido el LED rojo como alerta.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* Tiempo maximo de espera por linea en cada iteracion (ms) */
  const uint32_t READ_TIMEOUT_MS = 1000;

  uint8_t b;
  int got = LORA_PollAndGetByte(&b, READ_TIMEOUT_MS);

  if (got == 1) {
    /* Tenemos un byte decodificado del primer par HEX ("30".."39" -> '0'..'9') */
    char to_show;
    if (b >= '0' && b <= '9') {
      to_show = (char)b;                /* Maestro envia ASCII '0'..'9' */
    } else if (b <= 9) {
      to_show = (char)('0' + b);        /* soporte 00..09 crudo */
```

```
18      } else {
19        to_show = '.';
20      }
21
22      /* Mostrar en la OLED (helper de la subseccion OLED)
            */
23      OLED_ShowLastDigit(to_show);
24
25      /* Feedback visual: LED verde = RX OK */
26      HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12);
27      /* Asegurar LED rojo apagado cuando hay exito */
28      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET
           );
29    }
30    else if (got < 0) {
31      /* Linea "radio_rx ..." pero mal formada (ya
           rearmamos RX) */
32      HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
             /* LED rojo = alerta */
33    }
34
35    /* Aqui puedes incluir otras tareas no bloqueantes si
         las necesitas */
36  }
37  /* USER CODE END WHILE */
```

Listing 24: While del Esclavo: recepcion, decodificacion y OLED

Que esta pasando

- **Lectura no bloqueante:** `LORA_PollAndGetByte()` usa `LORA_ReadLine()` con un timeout corto (1 s por vuelta). Si no hay nada, devuelve 0 y el while sigue fluido.
- **Decodificacion HEX:** del mensaje `radio_rx <hex...>` tomamos solo el **primer par HEX** y lo convertimos a byte. Si el Maestro envia ASCII `'0'..'9'`, veras los digitos en la OLED.
- **Rearmado automatico:** despues de cada evento (rx/err/busy), el helper rearma internamente `radio rx 0`.
- **Diagnostico por LEDs:** verde (`PD12`) indica recepcion correcta; rojo (`PD14`) marca linea mal formada.

Prueba rapida

1. En el Maestro, confirma que estas enviando `"30".."39"` (ASCII de `'0'..'9'` en HEX).

2. En el Esclavo, deberias ver el digito recibido en la OLED y el LED verde parpadear cada paquete.

3. Si no recibes nada: revisa que `freq/sf/bw/cr/crc/iqi/prlen` coincidan, antena conectada, y que el Maestro no este demasiado cerca (saturacion).

4. Resultados

En esta sección se presentan evidencias de funcionamiento y métricas simples del enlace Maestro–Esclavo usando **LoRa RN2483** en modo *radio* (punto a punto). El Maestro transmite un dígito ASCII (`'0'..'9'`) cada 1 s; el Esclavo recibe, decodifica (*HEX ASCII*) y muestra el carácter en la **OLED SSD1351**. Se comparan además ambas bandas de operación (**433 MHz** y **868 MHz**).

4.1. Imágenes del funcionamiento

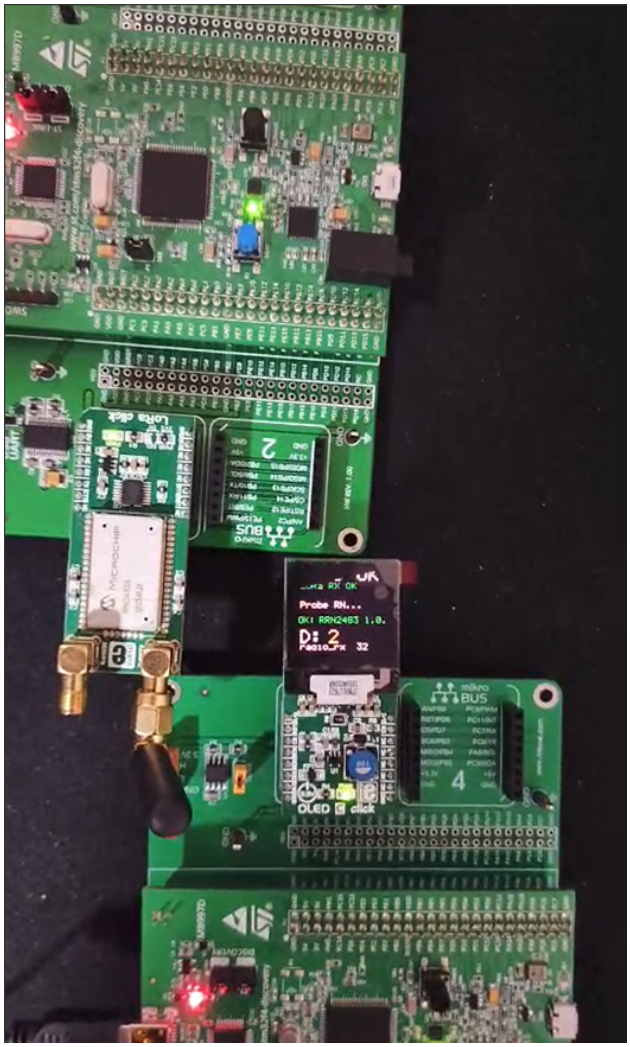


Figura 12: Esclavo recibiendo: OLED mostrando el dígito decodificado (ej. '7') y mensaje "LoRa RX OK".

4.2. Diferencias entre frecuencias (433 MHz vs 868 MHz)

Penetración y pérdidas A igual distancia, la banda de 433 MHz sufre **menor pérdida en espacio libre** que 868 MHz (por la dependencia con la frecuencia). Usando la fórmula de path loss en espacio libre (FSPL, dB):

$$\mathrm{FSPL} = 32{,}44 + 20\log_{10}(f_{\mathrm{MHz}}) + 20\log_{10}(d_{\mathrm{km}})$$

se tienen ejemplos didácticos:

Cuadro 4: FSPL teórica comparada (referencia)

Distancia	**433 MHz (dB)**	**868 MHz (dB)**	**(868-433)**
10 m	45.17	51.21	$\approx 6{,}0$
50 m	59.15	65.19	$\approx 6{,}0$
100 m	65.17	71.21	$\approx 6{,}0$

Esto explica que, en general, **433 MHz** tiende a comportarse mejor en interiores u obstáculos (mayor longitud de onda, mejor difracción/penetración). Por su parte, **868 MHz** usa antenas más cortas y, combinado con BW amplias y SF bajos, puede ofrecer menor *time-on-air* (menor latencia) en escenarios de corto alcance.

Tamaño de antena (aprox. cuarto de onda)

$$\lambda = \frac{c}{f} \Rightarrow \lambda_{433} \approx 0{,}693\,\text{m}\ (\lambda/4 \approx 17{,}3\,\text{cm}), \quad \lambda_{868} \approx 0{,}346\,\text{m}\ (\lambda/4 \approx 8{,}6\,\text{cm})$$

Antenas más cortas en 868 MHz son prácticas para montajes compactos; 433 MHz favorece enlaces más robustos con antenas algo más largas.

Parámetros físicos (SF/BW/CR) y ToA Con BW=125 kHz y CR=4/5, el **tiempo en aire** para un **payload de 1 byte** es, teóricamente:

- SF7: $\approx 25{,}9$ ms
- SF9: $\approx 103{,}4$ ms
- SF12: $\approx 827{,}4$ ms

Reducir SF (p.ej., SF7) y/o aumentar BW (p.ej., 250/500 kHz) disminuye el ToA y la latencia, a costa de alcance/robustez. **Maestro y Esclavo deben tener la misma PHY** (freq/SF/BW/CR/CRC/IQ/PRLEN).